



Waipapa
Taumata Rau
**University
of Auckland**

CS340 – Tutorial Week 4

Scheduling Recap & Assignment 1 Part 2: Priority Scheduler

➤➤➤ August 26

➤➤➤ Connor Williams

Plan for today



We'll recap this week's lectures, then spend some time on assignment 1: Part 2, the xv6 priority scheduler.

Quick recap: IPC — pipes & signals

Quick recap: scheduling — FCFS to MLFQ

Quick recap: virtual memory & paging

Assignment 1 – Part 2: priority scheduler deep dive

Practice questions & working time



Attendance

Quick Recap: This Week's Lectures

IPC · Scheduling · Virtual memory & paging

IPC: Two Ways Processes Talk



Each process has its own address space. IPC creates a controlled path across that boundary.

Data

A pipe moves bytes from one process to another

Notification

A signal tells a process that an event occurred

Part 1 of Assignment 1 (pingpong, findppid, primes) is built entirely on `fork()`, `pipe()`, and `wait()`

A Pipe Is a Kernel-Managed Byte Stream

`int fd[2]; pipe(fd);` — `fd[0]` is the read end, `fd[1]` is the write end

The kernel holds written bytes until the reader receives them, in order

A pipe is a stream — it does not preserve message boundaries

`fork()` after `pipe()` gives parent and child copies of both descriptors

- Each process closes the end it doesn't use, then reads or writes the other

`read()/write()` return how many bytes actually moved — always check, and repeat the call if a larger transfer needs several rounds

Regular Pipes vs Named Pipes

Regular pipe

Created with `pipe()`

Reached through descriptors inherited across `fork()`

Typical pair: parent and child

This is what pingpong and primes use

Named pipe

Created with `mkfifo("path", mode)`

Reached by opening the same filesystem pathname

Typical pair: unrelated processes started separately

Not used in Assignment 1, but good to recognise

Signals Deliver Events, Not Data

The kernel delivers a signal to tell a process something happened

- Ctrl+C → SIGINT · bad memory access → SIGSEGV · child state change → SIGCHLD

A process can accept the default action, ignore many signals, or run a handler

SIGKILL and SIGSTOP cannot be caught or ignored

Handlers run outside normal control flow — keep them small and stick to safe operations such as write and _exit

Scheduling Deep Dive

From FCFS to a real xv6 priority scheduler



A Scheduler Combines a Choice With a Context Switch

Policy

Select the next ready process

Set how long it may run

Prioritise the desired metric

Mechanism

Save the current CPU state

Select another saved state

Restore it safely

Timer interrupts let the kernel preempt a process that never yields voluntarily

FCFS, SJF, SRTF, and Round-Robin Compared

No policy optimises waiting, response, throughput, overhead, and fairness all at once — that trade-off is the whole point.

Policy	Preemptive?	Strength	Weakness
FCFS	No	Simple, low overhead	Convoy effect — a long job delays every short job behind it
SJF	No	Minimises average wait, given accurate burst estimates	Needs burst predictions; can starve long jobs
SRTF	Yes	Preempts for a shorter arrival, lowers average wait further	Still needs accurate remaining-time estimates
Round-Robin	Yes (quantum)	Fair turns, bounded response time	Quantum choice trades response time vs context-switch overhead

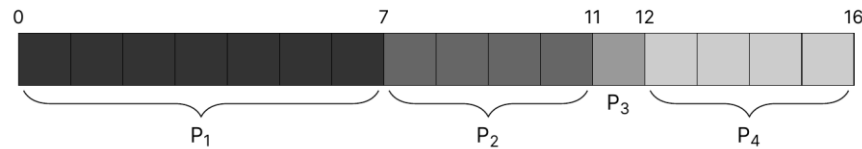
Same Four Processes, Four Policies

FCFS

Consider the following processes:

Process	Arrival Time	Burst Time
P ₁	0	7
P ₂	0	4
P ₃	0	1
P ₄	0	4

Assume, P₁ → P₂ → P₃ → P₄. For FCFS, our schedule is:

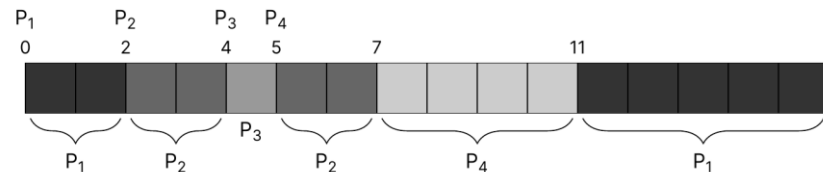


SRTF

Consider the same processes and arrival times as SJF:

Process	Arrival Time	Burst Time
P ₁	0	7
P ₂	2	4
P ₃	4	1
P ₄	5	4

For SRTF, our schedule is (arrival on top):

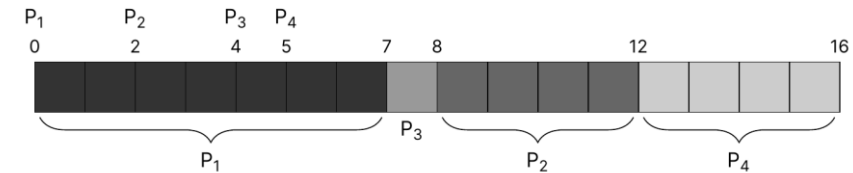


SJF

Consider the same processes with different arrival times:

Process	Arrival Time	Burst Time
P ₁	0	7
P ₂	2	4
P ₃	4	1
P ₄	5	4

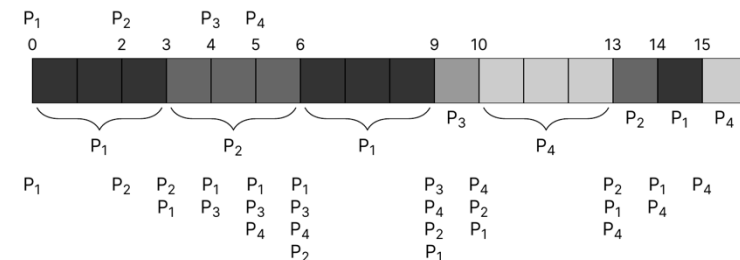
For SJF, our schedule is (arrival on top):



Round-Robin (quantum 3)

Process	Arrival Time	Burst Time
P ₁	0	7
P ₂	2	4
P ₃	4	1
P ₄	5	4

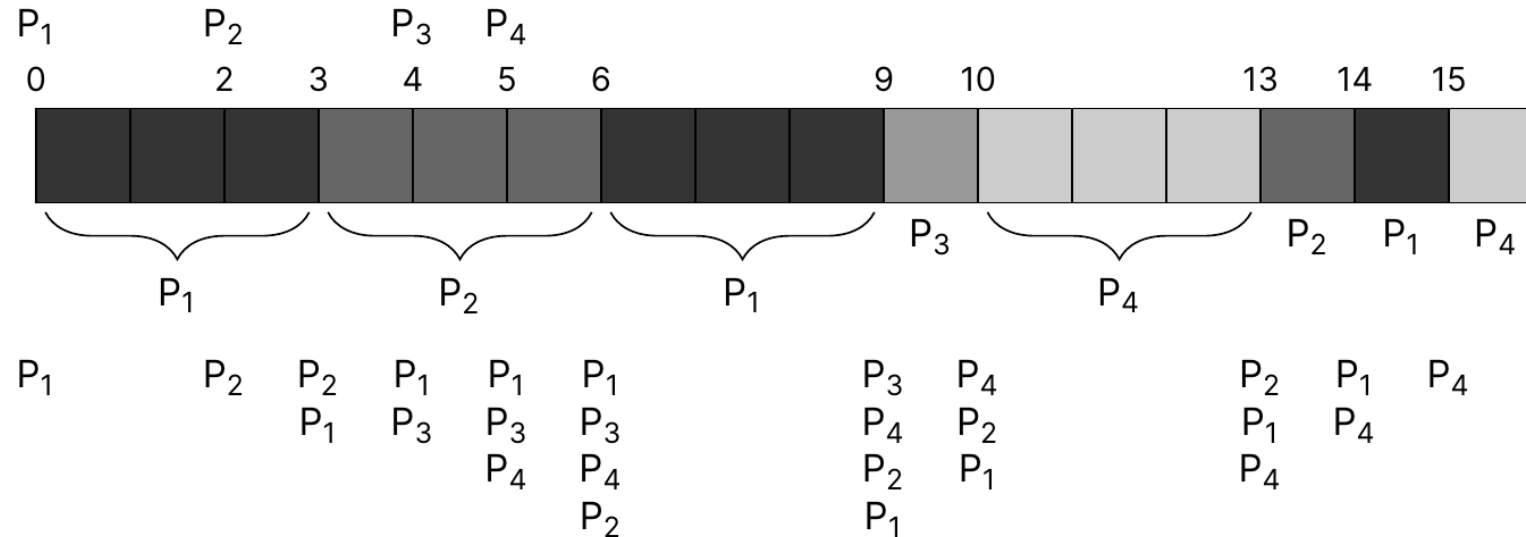
For RR, our schedule is (arrival on top, queue on bottom):



Worked Example: Round-Robin, 3-Unit Quantum

Process	Arrival Time	Burst Time
P ₁	0	7
P ₂	2	4
P ₃	4	1
P ₄	5	4

For RR, our schedule is (arrival on top, queue on bottom):



7 context switches · average waiting time 7 · average response time 2.75

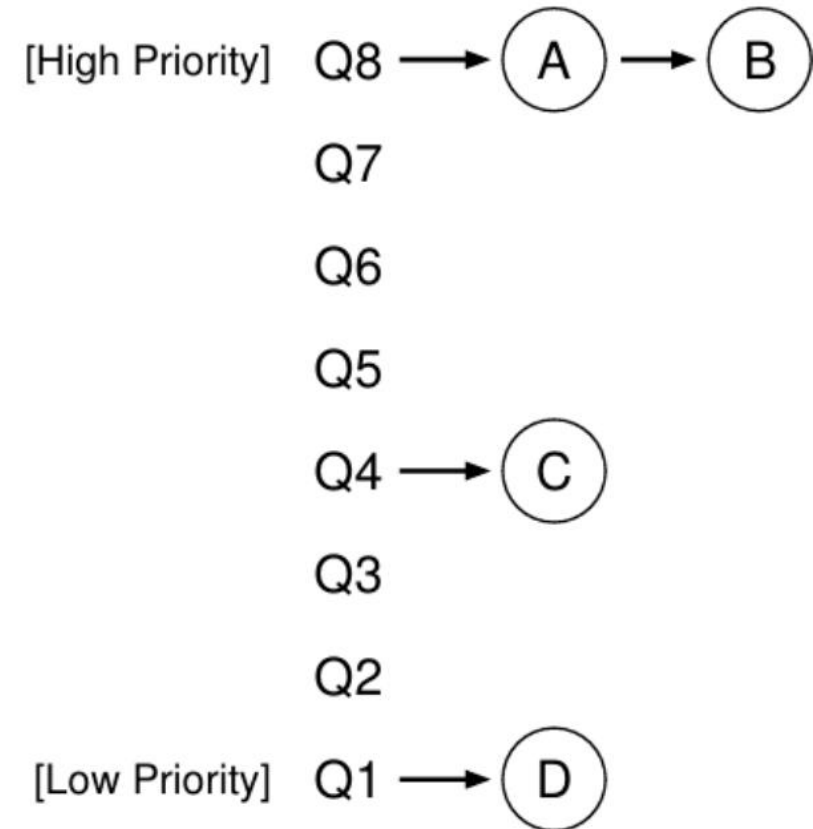
MLFQ Learns From How a Process Uses the CPU

A Multi-Level Feedback Queue keeps several ready queues, high to low priority

A process in a higher queue always runs before a lower one; processes in the same queue round-robin

New/interactive jobs start high; sustained CPU use moves a job down; periodic boosts stop starvation

This is exactly the shape of Part 2's scheduler — just simplified to two fixed levels that a process sets for itself with `setpri`, no automatic demotion



One MLFQ snapshot: eight ready queues, high to low priority

Beyond Two Levels: Real-Time and Fair Scheduling

Real-time policies (Linux SCHED_FIFO / SCHED_RR) run the highest static priority first — the same rule Part 2 asks you to implement

Normal/fair scheduling (CFS, EEVDF) instead tracks CPU service so every runnable task gets a proportional share

Part 2's scheduler is a tiny two-level version of a real-time class: high priority always wins, ties are broken round-robin

Assignment 1 – Part 2: Priority Scheduler

The main kernel task

The Task: Replace xv6's Scheduler



Part 2 is the main kernel task in Assignment 1 — you'll modify core scheduling logic in `kernel/proc.c` (C) or `kernel/src/proc.rs` (Rust).

Every process is assigned a priority: 1 (low) or 2 (high)

All processes must start at priority 1 when created

A runnable priority-2 process must always run before any priority-1 process

A priority-1 process must not run while any priority-2 process is runnable

Processes that share a priority round-robin between them

Two New System Calls Drive the Scheduler

setpri

`int setpri(int num)` — changes the calling process's priority

Return -1 for anything other than 1 or 2

Return 0 on success

Rust: `fn setpri(num: usize) → usize`, returning an `Err/Ok Result`

getpinfo

`int getpinfo(struct pinfo *)` — reports each process's PID and ticks spent at each priority level

Return -1 for a bad pointer, 0 on success

Use it to build a tiny `ps` and check your scheduler is actually obeying the rules

You've Already Wired a Syscall: Task 3's getppid

Add the syscall number and prototype (kernel/syscall.h, kernel/syscall.c, or the Rust syscall table)

Declare it for user space (user/user.h + user/usys.S, or user/src/syscall.rs)

Implement the kernel-side function

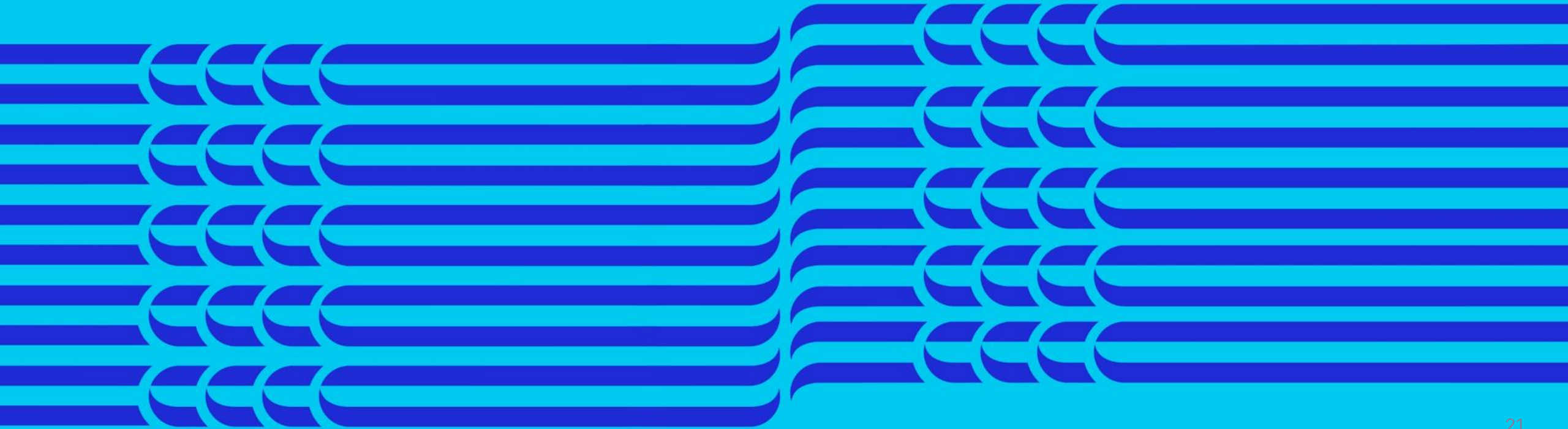
setpri and getpinfo follow the exact same checklist — the only new part is that they read and write fields on struct proc directly, instead of just returning a single value

Common Pitfalls in Part 2

- X - Forgetting `allocproc()` must default every new process to priority 1
- X - Checking priority once instead of re-checking after every context switch
- X - Breaking round-robin fairness by always restarting the scan from the front of the process table
- X - Forgetting the single-CPU change: `CPUS := 1` (C Makefile) or `NCPU: usize = 1` (Rust `params.rs`)
- X - Not returning `-1 / Err` for a priority that isn't 1 or 2

Practice Questions

Work through these with the person next to you



Practice 1: Trace This Priority Schedule

Process	Arrival	Burst	Priority
A	0	6	1
B	2	2	2
C	4	3	2
D	5	4	1

Using Part 2's rules (priority 2 always before priority 1, round-robin within a level) answer:

Which process is running at time 4?

Which process is running at time 7?

Does D ever run before B and C finish? Why or why not?

Sketch the full schedule from $t=0$ to completion.

Practice 1: Trace This Priority Schedule

Process	Arrival	Burst	Priority
A	0	6	1
B	2	2	2
C	4	3	2
D	5	4	1

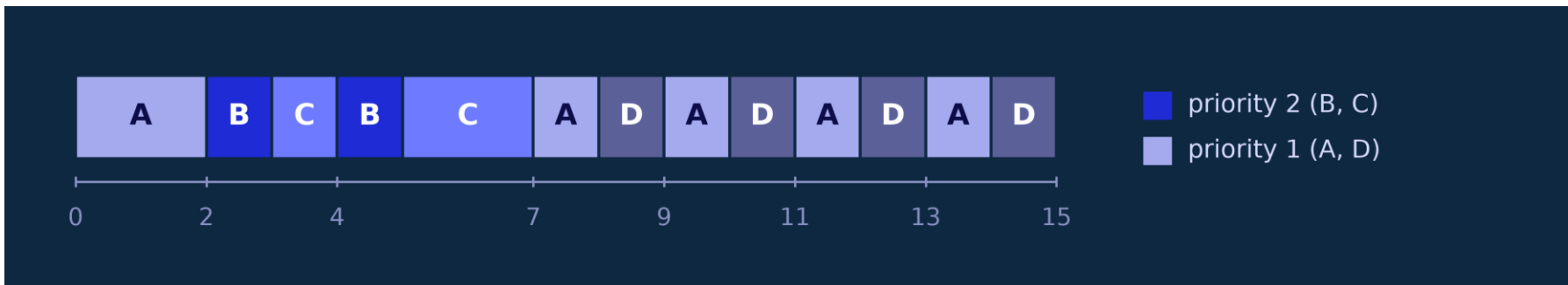
Using Part 2's rules (priority 2 always before priority 1, round-robin within a level) answer:

Which process is running at time 4?

Which process is running at time 7?

Does D ever run before B and C finish? Why or why not?

Sketch the full schedule from $t=0$ to completion.





Practice 2: Find the Bug

```
int setpri(int num) {  
    myproc()->priority = num;  
    return 0;  
}
```

This compiles, and casual testing with setpri(2) 'works'.

Which rule(s) from the spec does it silently break?
What should the function do differently?



Practice 3: Talk It Through

Two priority-1 processes are round-robinning. A priority-2 process becomes runnable mid-quantum. What exactly must happen, and when?

Why must every process start at priority 1, never priority 2?



Waipapa
Taumata Rau
**University
of Auckland**

Questions?